

FPL Player Analysis

Project explainer, script guide, terminology sheet and field glossary

A Python + SQL pipeline that turns the free Fantasy Premier League API into a weekly decision report for entry **234865** — and a portfolio-grade example of a layered data warehouse built on a laptop.

Version 1.0 · 17 September 2026 · Season 2026/27

Prepared for Angus Macdonald by Scout

Repository folder: `Personal Projects / FPL Player Analysis`

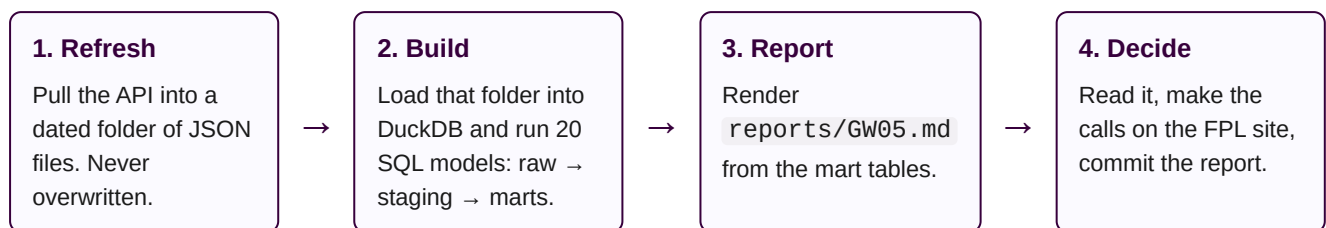
Contents

1. What this project is, in one page
2. How the pieces fit together
3. The scripts, one by one
4. The SQL models, one by one
5. The scoring model, explained plainly
6. How to read the weekly report
7. The weekly routine and every command
8. Configuration reference
9. Terminology sheet — FPL terms
10. Terminology sheet — data and engineering terms
11. Field glossary — the columns you will actually query
12. Raw API fields — units and gotchas
13. Troubleshooting
14. Known limitations and where the plan goes next

1. What this project is, in one page

Every week of the Premier League season you make four Fantasy Premier League decisions: who to captain, whether to transfer anyone, who sits on the bench, and whether to play a chip. This project makes those decisions *evidence-first*. It pulls every player's numbers from the official FPL API, stores them in a small local database, runs a set of SQL models that turn raw statistics into a modelled "expected points" figure for each player's next five fixtures, and prints a Markdown report ranking captains, transfers and value picks for your specific squad.

The API is free, public and needs no login — it is the same data the FPL website itself reads. Your squad, bank and transfer history are public too (anyone with your team ID can see them on the site), which is why the tool can be squad-aware without ever asking for a password.



Three ideas underpin everything, and knowing them makes the rest of the code obvious:

- **Snapshots, not overwrites.** The API only shows *now*. Prices, ownership and injury flags change hourly and the old values vanish. So every refresh writes a new timestamped folder and never touches an old one. Anything about change over time is built from those folders.
- **Raw → staging → marts.** The same three-layer pattern used in a Snowflake/dbt warehouse. Raw keeps the API's own names and types. Staging renames and casts, once. Marts hold the analytics. Business logic lives only in marts, so a change to the API is a one-line staging fix, never a rewrite.
- **The report is the product and the decision log.** Every number in it traces back to a row in a mart table. The report files are committed to Git, so at season's end you can audit what the model said in GW12 and what you did.

Why DuckDB and not Snowflake? Because a personal project has to survive 38 weeks and trial accounts expire. DuckDB is a single file, zero cost, reads JSON natively, and its SQL dialect is close enough to Snowflake's that the model files are written Snowflake-style and would lift across with only the raw-load step changing. Phase 4 of the project plan is exactly that port.

2. How the pieces fit together

The folder map

Path	What lives there
config/squad_override.toml	Your real squad for the upcoming gameweek, used instead of the API picks when <code>enabled = true</code> (the API hides pre-deadline picks — see section 12).
config/settings.toml	Every tunable: your entry ID, the model weights, the horizon, file paths. No secrets — the API needs none. Any value can be overridden by an environment variable <code>FPL_<SECTION>_<KEY></code> .
src/fpl_analysis/	The Python package: seven modules described in section 3, plus the report template.
sql/staging/	Ten SQL files, one view each. Rename, cast, unpivot. No business logic.
sql/marts/	Nine SQL files, one table each. Fixture outlook, the expected-points model, your squad, captaincy, transfers, differentials.
data/raw/<snapshot_id>/	One folder per refresh, holding the verbatim API JSON and a <code>manifest.json</code> . Git-ignored.
data/fpl.duckdb	The warehouse file. Rebuilt from the latest snapshot on every <code>fpl build</code> . Git-ignored.
reports/GWxx.md	The weekly decision reports. Committed to Git.
notebooks/	Three Jupyter notebooks for VS Code: explore the API, analyse players in SQL, run the weekly routine.
docs/	PROJECT_PLAN, ARCHITECTURE, SCORING_MODEL, FPL_API_REFERENCE, DATA_DICTIONARY — and this explainer.
tests/	Fourteen pytest tests that run against a synthetic season (no network), plus the generator that builds it.
.github/workflows/ci.yml	GitHub Actions: lint and test on every push, Python 3.11 and 3.12.

The four layers of the warehouse

Schema	Kind	What it holds	Rule
raw	tables	One table per API array from the latest snapshot: <code>players</code> , <code>teams</code> , <code>events</code> , <code>fixtures</code> , <code>player_history</code> , <code>entry_picks</code> and so on. Column names are the API's own. Numbers the API sends as strings stay strings.	Never edited. Replaced wholesale on every build.
history	table	<code>player_snapshot</code> : one row per player per refresh — price, ownership, form, status. The only place data from <i>earlier</i> refreshes survives.	Append-only. Idempotent: re-loading the same snapshot adds nothing.
staging	views	One view per raw table with clean names and proper types (<code>stg_players</code> , <code>stg_fixtures</code> ...). <code>stg_team_fixtures</code> unpivots fixtures to one row per team per match.	Rename and cast only. No joins across business concepts, no calculations.

Schema	Kind	What it holds	Rule
marts	tables	The analytics: mart_player_expected_points, mart_player_horizon, mart_squad, mart_transfer_candidates ...	All the business logic. Materialised as tables so the report reads frozen numbers.

Model lineage (what feeds what)

```

raw.players —————> stg_players ————┐
raw.teams —————> stg_teams —————┐→ mart_player_form →
mart_player_expected_points → mart_player_horizon
raw.element_types —→ stg_positions ————┐          ▲          ▲
|
raw.player_history —→ stg_player_gameweek_history ———┐          |
└→ mart_captaincy
raw.fixtures —————> stg_fixtures → stg_team_fixtures → mart_team_fixture_outlook →
mart_team_fixture_summary
raw.snapshot —————> stg_snapshot —————┐
└→ mart_differentials
raw.entry_picks —————> stg_entry_picks
└────────────────────────────────────────┐→ mart_squad →
mart_transfer_candidates
raw.entry_gameweeks → stg_entry_gameweeks (used directly by the report for standing +
free-transfer estimate)

```

Read it left to right. A change to anything on the left ripples right; nothing flows the other way. When a number in the report looks wrong, walk left along its row until you find the table where it first goes wrong — that is the file to fix.

3. The scripts, one by one

All Python lives in `src/fpl_analysis/`. Each module has one job. They are listed in the order data flows through them.

config.py — settings

What it does. Reads `config/settings.toml`, applies any `FPL_*` environment-variable overrides, resolves the file paths against the project root, validates that the three model weights sum to 1.0, and hands back a frozen `Settings` object that every other module receives as its first argument.

How to interpret it. If a behaviour is tunable, it is in here and nowhere else. `find_project_root()` walks up from the current directory until it finds `pyproject.toml`, which is why the notebooks work from `notebooks/` and the CLI works from anywhere inside the repo.

When you would touch it. Only to add a new setting. Add the key to the TOML, add a field to `Settings`, wire it in `load_settings()`.

api.py — the FPL client

What it does. A thin wrapper around `requests` with one method per endpoint: `bootstrap_static()`, `fixtures()`, `element_summary(player_id)`, `entry(id)`, `entry_history(id)`, `entry_picks(id, gw)`, `entry_transfers(id)`. It sends a browser-like User-Agent (the default Python one is sometimes blocked by Cloudflare), retries transient failures with exponential back-off (1 s, 2 s, 4 s), and raises `FPLAPIError` when it gives up. `current_and_next_gameweek()` reads the `is_current / is_next` flags from the gameweek list.

How to interpret it. It returns parsed JSON and nothing else. It deliberately knows nothing about what the fields mean — that keeps a change in the API's shape a staging-SQL fix rather than a client rewrite.

ingest.py — raw snapshots

What it does. `create_snapshot()` makes a folder named by UTC timestamp (`20260917_084552`), pulls bootstrap and fixtures, pulls your four entry endpoints, *writes the manifest immediately* so the snapshot is usable, then loops over every player calling `element-summary` (about 660 calls, 0.15 s apart, three to four minutes) and writes the per-player history, past seasons and upcoming fixtures as three flat JSON arrays. `list_snapshots()`, `latest_snapshot()` and `get_snapshot()` read manifests back.

How to interpret it. A snapshot folder is the unit of reproducibility: `fpl build --snapshot 20260917_084552` rebuilds exactly what you saw that day. The manifest records which gameweek was current and next at pull time, so a build is anchored to the gameweek in the data, not the clock on your wall.

Safety behaviour. If the per-player loop is interrupted (Ctrl-C) the snapshot is kept without history and still builds. If five consecutive player calls fail, the loop stops — that pattern means the API is refusing us, and each failed call already costs seven seconds of retries. Progress logs every 50 players.

store.py — the warehouse and the model runner

What it does. Four things. `connect()` opens `data/fpl.duckdb` and ensures the four schemas exist. `load_snapshot()` reads each JSON file straight into a `raw.*` table with DuckDB's JSON reader (`read_json_auto + UNNEST`), stamping every row with the `snapshot_id`; optional files that are absent get an empty, correctly typed table so every view still compiles. `append_history()` copies the key per-player facts into `history.player_snapshot` with `INSERT OR IGNORE`. `run_models()` discovers every `.sql` file under `sql/`, renders it as a Jinja template, and executes `CREATE VIEW|TABLE schema.name AS ...` in filename order.

How to interpret it. The runner is a 60-line stand-in for dbt. Two Jinja helpers mirror dbt's so a later migration is mechanical: `{{ ref('stg_players') }}` resolves to the qualified table name, and `{{ var('horizon_gameweeks') }}` pulls a value from settings. A model whose file begins `-- materialized: table` becomes a table; otherwise staging models are views and marts are tables. The numeric filename prefixes (`010_`, `100_`) set the execution order explicitly.

Also here. `build()` = load + append history + run models, the thing `fpl build` calls. `query()` opens a read-only connection, runs one SQL string and returns a pandas DataFrame — the notebook workhorse.

squad_override.py — your real squad before the deadline

What it does. Reads `config/squad_override.toml`; when enabled, resolves the 15 names against `raw.players` (qualify a duplicate name with `{ name = "Palmer", position = "GKP" }`), and replaces `raw.entry_picks` and the bank with what you stated, stamping `source = 'manual override'`. Ambiguous or unknown names fail loudly with the candidates listed.

Why it exists. The public API only reveals a gameweek's picks after its deadline, so without this the deadline-day report would model transfers against last week's squad.

Transfer accounting. The build keeps the untouched API picks in `raw.entry_picks_api`. When the override is active the report diffs the two squads, lists the transfers made this week (in / out), and turns the free-transfer count going into the gameweek (the estimate, or `free_transfers` in the override file) into "free transfers left" and "hit taken" (4 points per move beyond the free ones; Wildcard and Free Hit weeks are free). The transfer section then tells you how many further moves are free before a hit applies.

squad.py — free-transfer estimate and pick helpers

What it does. `estimate_free_transfers()` reconstructs how many free transfers you have banked, because the public API never exposes that number. It replays your gameweek history with the rules in force since 2024/25: start with 1 after your first gameweek, +1 per gameweek capped at 5, transfers consume the bank first, a Wildcard or Free Hit week leaves the bank untouched. `picks_summary()` splits the 15 picks into starters, bench, captain and vice.

How to interpret it. The report labels the figure "estimated". If it ever disagrees with the site, set `free_transfers_override` in settings and tell me — it means a rule changed.

optimiser.py — Best Free Hit and Best Wildcard squads

What it does. Picks the best *legal* 15 for each chip by solving a small integer programme (SciPy's bundled HiGHS solver, under a second): 2/5/5/3 in the squad, a valid starting XI, at most three per club,

within your budget (squad value + bank), maximising modelled EP with the captain doubled plus a small weight on the bench. Free Hit uses `ep_next` (one week); Wildcard uses `ep_horizon` (five weeks) with a heavier bench weight, because a Wildcard squad has to survive rotation. Writes `marts.mart_chip_squads`.

How to interpret it. This is the one piece of analytics written in Python rather than SQL, because SQL has no solver; it still reads only from a mart and writes only to a mart. The report shows each squad's XI EP next to your current XI on the same basis, so the *gain* is the decision number, and marks which players you already own so the number of Wildcard changes is visible. The answer is exact, not a heuristic — but it inherits every limitation of the EP model underneath it.

report.py — the gameweek report

What it does. `build_report()` reads the mart tables into DataFrames, converts each to a Markdown table with the report's column headings, computes a few scalars (deadline in your timezone, free-transfer estimate, modelled EP of your current XI), and renders `templates/gameweek_report.md.j2` to `reports/GW<next>.md`.

How to interpret it. Every table in the report is a straight projection of one mart. Section 6 of this document maps each report column back to its source column. To change the layout, edit the template; to change a number, edit the mart.

cli.py — the commands

What it does. Defines `fpl refresh | build | report | run | snapshots | history-rebuild | query` with argparse and dispatches to the modules above. `python -m fpl_analysis ...` is identical. See section 7 for every flag.

templates/gameweek_report.md.j2 — the report layout

A Jinja template. Variables such as `{{ starters_md }}` are the pre-rendered Markdown tables; `{% if has_squad %}` hides the squad sections when a snapshot was taken with `--no-entry`. It ends with a "How to read the numbers" footer that repeats the weights actually used.

tests/ — the safety net

`tests/synthetic.py` fabricates a whole season — 20 clubs, 120 players, 380 fixtures, four gameweeks of history, a legal 15-man squad for entry 234865 — using the exact field names of the real API (verified live on 17 September 2026). The 14 tests in `test_config.py`, `test_squad.py` and `test_pipeline.py` run the full pipeline against it in a couple of seconds with no network. `pytest` runs them; CI runs them on every push.

notebooks/ — where you work in VS Code

Notebook	Needs	Use it to
01_explore_api.ipynb	network only	See what each endpoint returns before trusting the pipeline. Shows the string-typed numbers, the FDR direction, your squad straight from the API.
02_player_analysis.ipynb	a built warehouse	Write SQL against the marts: top players by EP, model-vs-API comparison chart, fixture swings, price history, scratch queries.

Notebook	Needs	Use it to
03_weekly_decision.ipynb	network + warehouse	The Friday routine as cells: refresh → build → report, render the report inline, then drill into the captaincy and transfer evidence and run a what-if on the weights.

The first cell of every notebook adds `src/` to `sys.path` and loads settings; run it first. Notebooks stay thin by design — logic lives in `src/` and `sql/`, notebooks call it.

4. The SQL models, one by one

Every model file starts with a comment stating its grain ("one row per ...") and its purpose. This section says what each does and how to read its key columns. Full column lists are in section 11.

Staging (views)

Model	Grain	What it does and how to read it
stg_snapshot	one row	The context of the loaded snapshot: <code>current_gameweek</code> , <code>next_gameweek</code> , when it was pulled. Every mart anchors "upcoming" on <code>next_gameweek</code> from here, which is what makes a build reproducible from any old snapshot.
stg_teams	one per club	Clean names (<code>team_short_name</code> = ARS, LIV...), FPL's strength ratings, league record. <code>matches_played</code> is used downstream to turn a player's <code>starts</code> into a start share.
stg_gameweeks	one per GW	Deadlines cast to real timestamps, the <code>is_current</code> / <code>is_next</code> / <code>is_finished</code> flags, and the most-captained / most-transferred player IDs.
stg_positions	one per position	GKP / DEF / MID / FWD with the squad rules (2/5/5/3 in a squad; min and max starters).
stg_players	one per player	The most important staging model. The API sends <code>form</code> , <code>points_per_game</code> , <code>selected_by_percent</code> , all the xG fields and <code>ep_next</code> as <i>strings</i> ; this view casts them all to DOUBLE with <code>TRY_CAST</code> , once. It also converts money from tenths to millions (<code>price_m</code>) and renames the API's terse names (<code>element_type</code> → <code>position_id</code>).
stg_fixtures	one per fixture	380 rows. <code>gameweek</code> is NULL for a postponed match not yet rescheduled. <code>home_team_fdr</code> is the difficulty faced <i>by the home team</i> ; <code>away_team_fdr</code> by the away team — easy to get backwards.
stg_team_fixtures	one per team per fixture	The same fixtures unpivoted: 760 rows, each from one team's point of view with <code>opponent_team_id</code> , <code>is_home</code> and <code>fdr</code> already oriented. Double gameweeks (two rows for a team in one GW) and blanks (none) fall out naturally from this shape.
stg_player_gameweek_history	one per player per fixture played	This season's per-match rows: points, minutes, per-match xG/xA/xGC, price at the time, ownership count. Empty if the refresh ran with <code>--no-history</code> ; marts LEFT JOIN it, so nothing breaks.
stg_player_history_past	one per player per past season	Career totals in the FPL era — priors for players with few minutes this season (unused by the model so far).
stg_entry_picks	one per pick (15)	Your squad for the current gameweek: <code>squad_position</code> 1–11 is the starting XI in formation order, 12–15 the bench order; captain/vice flags; <code>bank_m</code> and <code>squad_value_m</code> repeated on every row for convenience.

Model	Grain	What it does and how to read it
<code>stg_entry_gameweeks</code>	one per GW you have played	Your points, ranks, bank, transfers made and points left on the bench each week. Feeds the free-transfer estimate and the standing table.

Marts (tables)

Model	Grain	What it does and how to read it
<code>mart_team_fixture_outlook</code>	one per team per upcoming fixture in the horizon	Filters <code>stg_team_fixtures</code> to the next N gameweeks (N = <code>horizon_gameweeks</code> , default 5). Adds <code>horizon_index</code> (1 = next GW), the <code>fdr_multiplier</code> from settings, a <code>home_multiplier</code> (1.04 home, 0.96 away) and a human label like <code>LIV (A)</code> .
<code>mart_team_fixture_summary</code>	one per team	Rolls the outlook up: <code>fixtures_in_horizon</code> , <code>avg_fdr</code> , <code>fixture_score</code> (sum of the multipliers — a double gameweek scores higher on purpose), <code>fixture_run</code> as a readable string, and <code>fixture_rank</code> . The "who has the best run?" table.
<code>mart_player_form</code>	one per player	Lays every model input side by side: <code>form_signal</code> (API form), <code>season_ppg</code> , the computed <code>xgi_points_rate</code> , <code>availability</code> and its components, the <code>is_injury_risk</code> flag, <code>rates_are_trusted</code> (minutes above the threshold), and last-5-match averages from history. If you want to understand why a player is rated, start here.
<code>mart_player_expected_points</code>	one per player per upcoming fixture	The model itself, applied fixture by fixture. Shows <code>base_rate</code> , <code>availability</code> , both multipliers and the resulting <code>expected_points</code> , plus the substituted inputs actually used (<code>form_used</code> , <code>xgi_used</code>). This is the audit trail: any EP figure in the report is a sum of rows here.
<code>mart_player_horizon</code>	one per player	Sums the fixture rows into <code>ep_next</code> (next GW, doubles count twice) and <code>ep_horizon</code> (all N GWs), adds <code>ep_horizon_per_m</code> , <code>rank_overall</code> , <code>rank_in_position</code> and the readable <code>fixture_run</code> . The table the report ranks from.
<code>mart_squad</code>	one per pick (15)	Your picks joined to <code>mart_player_horizon</code> : each player's EP, next fixture, injury flag, bank and value. Empty if the snapshot was taken with <code>--no-entry</code> .
<code>mart_captaincy</code>	one per player with a fixture next GW	Ranked by <code>ep_next</code> ; <code>ep_as_captain</code> doubles it; <code>in_squad</code> lets the report show the best option you own next to the best overall.
<code>mart_transfer_candidates</code>	one per (out, in) pair, top 5 per outgoing player	For each of your 15, the best like-for-like replacements under the FPL rules: same position, affordable with current bank, not already owned, available, no injury risk, three-per-club respected after the swap. Ranked by <code>ep_horizon_gain</code> . <code>bank_after_m</code> shows what would be left.
<code>mart_differentials</code>	top 25 low-ownership players	<code>mart_player_horizon</code> filtered to ownership below <code>differential_max_ownership</code> (10%) and available, ranked by <code>ep_horizon</code> . The "nobody has him" upside plays.

5. The scoring model, explained plainly

The model produces one number per player per upcoming fixture, called **expected points (EP)**. It is a ranking device: its scale is roughly FPL points, but its job is to order players and swaps, not to forecast a score.

$$\text{EP}(\text{player}, \text{fixture}) = \text{base_rate} \times \text{availability} \times \text{fdr_multiplier} \times \text{home_multiplier}$$

base_rate — how good is he, per match?

Three signals blended with the weights from settings (defaults 45 / 30 / 25):

$$\text{base_rate} = 0.45 \times \text{form_used} + 0.30 \times \text{season_ppg} + 0.25 \times \text{xgi_used}$$

Signal	Where it comes from	What it captures	Its weakness
form	API <code>form</code> : average points per match over the last 30 days	Recent output, role changes, hot streaks	Noisy (4–5 matches); zero if he hasn't played in 30 days
season ppg	API <code>points_per_game</code>	Stable output level	Slow to react; includes early-season noise
xGI rate	Computed from per-90 underlying stats (below)	<i>Process</i> rather than outcome — players creating chances keep scoring after a cold streak	Per-90 rates lie on small samples

Form gets the most weight because FPL is played week to week; ppg anchors against one big haul; xGI is the regression-to-the-mean signal that penalises lucky finishers and rewards unlucky ones. The split is a reasoned prior, not a fitted one — the Phase 2 backtest fits it.

The underlying-stats rate

Points per 90 minutes if the player played the whole match, from the 2025/26 scoring rules:

$$\begin{aligned} \text{xgi_points_rate} = & 2 \text{ (appearance, 60+ min)} + \text{xG}/90 \times \text{goal_points}[\text{position}] \text{ (GKP 10, DEF 6, MID 5, FWD 4)} \\ & + \text{xA}/90 \times 3 \text{ (assist)} + \exp(-\text{xGC}/90) \times \text{cs_points}[\text{pos}] \text{ (clean sheet: GKP 4, DEF 4, MID 1, FWD 0)} \\ & + \text{saves}/90 \div 3 \text{ (goalkeepers)} - \text{xGC}/90 \div 2 \text{ (GKP and DEF: -1 per 2 conceded)} \\ & + \min(\text{bonus} \div \text{starts}, 3) \text{ (bonus per start, capped)} \end{aligned}$$

$\exp(-\text{xGC}/90)$ is the Poisson probability of conceding zero goals when the expected number is $\text{xGC}/90$ — the standard clean-sheet approximation.

Two guards

- **Zero form is not bad form.** If `form` = 0 but the player has minutes this season, he simply hasn't played in 30 days. The model substitutes `season_ppg` so a returning player isn't buried.
- **Small samples.** The xGI rate is only trusted once a player has `min_minutes_for_rates` (180) minutes. Below that, its weight moves onto `season_ppg`.

availability — will he actually play?

```
availability = chance_factor × start_share × status_factor
chance_factor = chance_of_playing_next_round ÷ 100 (NULL means no news = 1.0)
start_share = min(1, starts ÷ team_matches_played)
status_factor = 0 if status is 'u' or 'n', else 1
```

This is the crude part and the first thing Phase 2 improves: it is harsh on new signings (few starts, many team games) and ignores substitute minutes. A player with chance of playing below 75% is additionally flagged `is_injury_risk` and excluded from transfer-in suggestions.

Fixture and venue

FDR	1 (easiest)	2	3	4	5 (hardest)
multiplier	1.25	1.15	1.00	0.85	0.70

Asymmetric on purpose: elite defences suppress returns more than weak ones inflate them. Home $\times 1.04$, away $\times 0.96$. A double gameweek is two rows that sum; a blank contributes nothing.

A worked example

A midfielder with form 6.0, season ppg 5.0, and underlying numbers of xG/90 0.35, xA/90 0.25, xGC/90 1.1, bonus per start 1.0, with 400 minutes played, fully fit, started every match, facing an FDR-2 fixture at home:

- $\text{xgi_points_rate} = 2 + 0.35 \times 5 + 0.25 \times 3 + e^{-1.1} \times 1 + 1.0 = 2 + 1.75 + 0.75 + 0.33 + 1.0 = \mathbf{5.83}$
- $\text{base_rate} = 0.45 \times 6.0 + 0.30 \times 5.0 + 0.25 \times 5.83 = 2.70 + 1.50 + 1.46 = \mathbf{5.66}$
- $\text{availability} = 1.0 \times 1.0 \times 1 = \mathbf{1.0}$
- $\text{EP} = 5.66 \times 1.0 \times 1.15 \times 1.04 = \mathbf{6.77}$ for that fixture

Sum his next five fixtures and you have his `ep_horizon`; the first of them is his `ep_next`.

6. How to read the weekly report

The report answers five questions in order. Read it top to bottom on deadline day.

§1 Where I stand

Team, overall points and rank, last gameweek's score and points left on the bench, squad value and bank, **free transfers (estimated)**, chips used, and the modelled EP of your current XI for the next gameweek (captain not doubled). The deadline is printed in your configured timezone. Then two tables:

Column	Source	Meaning
Pos / Player / Team / £m	<code>mart_squad</code>	Position, web name, club, current price in £m
Next	<code>next_fixture_label</code>	Next opponent and venue, e.g. LIV (A) . Blank means a blank gameweek
Form	<code>form_signal</code>	The API's own last-30-days average
xGI/90	<code>xgi_per_90</code>	Expected goal involvements per 90 — the cleanest attacking signal
Avail	<code>availability</code>	0–1. 0.50 means the model expects him on the pitch half the time. Below 1 for a starter is a prompt to check team news
EP next / EP 5GW	<code>ep_next</code> , <code>ep_horizon</code>	Modelled points next GW and across the horizon
Pos rank	<code>rank_in_position</code>	Where he ranks among all players in his position by EP over the horizon. A starter ranked 40th in his position is a transfer conversation

§2 Squad risks

Anyone in your 15 with `status` not "a" or a chance-of-playing below the threshold, with the news text. Empty is good. These players have their EP scaled by the chance and are excluded from transfer-in suggestions — but check again after the Friday press conferences, because the snapshot is only as fresh as your last refresh.

§3 Captaincy

Your current armband, then the best options *you own* ranked by `ep_next` with their overall rank alongside, then the best options overall. The "Own?" column tells you whether a top candidate is already in your squad. If the best owned option is ranked 14th overall while the top three are all players you could afford, that is a transfer signal as much as a captaincy one.

§4 Transfers

First the single highest-gain move for each outgoing player (top five by `ep_horizon_gain`), then the full shortlist grouped by outgoing player, weakest first. Columns:

Column	Meaning
Out / Out EP	Your player and his modelled EP over the horizon

Column	Meaning
In / In team / In £m / Own %	The candidate, his club, price and ownership
In fixtures	His run over the horizon, e.g. WHU (A), SUN (A), ARS (A), MCI (H), LEE (H)
In EP / EP next	The candidate's modelled EP over the horizon and next GW
Gain	<code>ep_horizon_gain</code> = In EP – Out EP. The decision number. A hit costs 4 points, so a hit is only worth it when Gain comfortably exceeds 4 <i>and</i> you believe the horizon
Bank after	What is left after the swap (using current price as selling price — see limitations)

§5 Best players by position

Top eight per position by `ep_horizon`, with the same columns as the squad table plus ownership and the full fixture run. This is the "what does the market look like" view; use it to sanity-check the model against your own eyes — if a name here surprises you, look him up in `mart_player_form`.

§6 Value and differentials

Best EP per £m lists the ten most efficient players who have enough minutes for their rates to be trusted — the enablers that free budget for premiums. **Differentials** lists low-ownership players the model rates: high upside relative to the field, higher variance.

§7 Fixture outlook by club

Every club ranked by `fixture_score` over the horizon, with games, home games, average FDR and the run. Doubles score higher by design. Use it to time transfers into a club rather than a player.

§8 Market movers

The eight players with the highest net transfers in this gameweek, with their EP. Useful for anticipating price rises (Phase 3 turns this into a proper price tracker).

§9 Best Free Hit team · §10 Best Wildcard team

The optimal legal 15 for each chip, with the XI EP (captain doubled), the same figure for your current XI, and the **gain** between them. A Free Hit is judged on next gameweek only; play it when the gain is large *and* the week is unusual (blanks, doubles, injuries), not for a few points in a normal week. A Wildcard is judged over the horizon; weigh its gain against what the number of changes would cost as hits, and remember the model says nothing about fixtures beyond the horizon. "Own?" marks players already in your squad.

The model is a ranking aid, not an oracle. It cannot see a manager's press conference, a tactical switch, or a player carrying a knock. Every call it makes should survive contact with team news and your own judgement before you act on it.

7. The weekly routine and every command

When	What	Command
Thursday evening	Full refresh once the pressers have started	<code>fpl refresh</code>
Friday (deadline day)	Build, report, read, decide, act on the FPL site	<code>fpl build && fpl report</code> (or just <code>fpl run</code> to refresh again first)
Friday	Commit the decision log	<code>git add reports/ && git commit -m "GW05 decisions"</code>
Any day (from Phase 3)	Cheap price snapshot around 02:30 UK	<code>fpl refresh --no-history</code>

Command reference

Command	What it does	Flags
<code>fpl refresh</code>	Pull the API into a new dated snapshot under <code>data/raw/</code> . About 3–4 minutes with player history.	<code>--no-history</code> skip the ~660 per-player calls (six calls total, seconds). <code>--no-entry</code> skip your squad endpoints.
<code>fpl build</code>	Load the latest snapshot into DuckDB, append to history, run all 20 models. Seconds.	<code>--snapshot ID</code> build a specific older snapshot.
<code>fpl report</code>	Render <code>reports/GW<next>.md</code> from the marts.	<code>--snapshot ID</code> ; <code>--stdout</code> print instead of writing a file.
<code>fpl run</code>	refresh + build + report in one go.	<code>--no-history</code> , <code>--no-entry</code>
<code>fpl snapshots</code>	List snapshots on disk with their current/next GW and whether they carry history.	
<code>fpl history-rebuild</code>	Replay every snapshot on disk into <code>history.player_snapshot</code> — after a fresh clone, say.	
<code>fpl query "SELECT ..."</code>	Run ad-hoc SQL against the warehouse and print a Markdown table.	<code>--limit N</code> (default 50)
global		<code>-v</code> debug logging; <code>--settings PATH</code> alternative settings file; <code>--version</code>

`python -m fpl_analysis <command>` is identical to `fpl <command>` if the script is not on your PATH (i.e. the venv isn't activated).

8. Configuration reference

All in `config/settings.toml`. Override any key per process with `FPL_<SECTION>_<KEY>=value`, e.g. `FPL_ANALYSIS_HORIZON_GAMEWEEKS=3`.

Key	Default	Effect
manager.entry_id	234865	Your FPL team ID (the number in the URL when viewing your team). Drives every squad-aware section.
manager.free_transfers_override	-1	-1 = estimate from history. Any other value is printed as-is in the report.
analysis.horizon_gameweeks	5	How many upcoming gameweeks EP is summed over. Shorter = more reactive, longer = favours fixture runs.
analysis.differential_max_ownership	10.0	Ownership % below which a player counts as a differential.
analysis.min_chance_of_playing	75	Below this, a player is an injury risk: flagged, excluded from transfer-in suggestions.
analysis.min_minutes_for_rates	180	Minutes before a player's own per-90 rates are trusted over his season ppg.
analysis.weight_form / weight_season_ppg / weight_xgi	0.45 / 0.30 / 0.25	The base-rate blend. Must sum to 1.0 (validated at load).
analysis.fdr_multiplier.1 ... 5	1.25 / 1.15 / 1.00 / 0.85 / 0.70	Fixture difficulty scaling.
chips.budget_override_m / bench_weight_freehit / bench_weight_wildcard / exclude_injury_risk	-1 / 0.05 / 0.15 / true	Chip-squad optimiser: budget (-1 = squad value + bank), how much bench EP counts, and whether flagged players are excluded.
report.timezone	Europe/Madrid	How the deadline is printed.
report.top_n_per_position	8	Rows in each "best by position" table.
report.captaincy_n	6	Rows in the overall captaincy table.
report.transfer_suggestions_n	3	Candidates shown per outgoing player in the full shortlist (the mart keeps 5).
paths.*	data/raw, data/fpl.duckdb, reports, sql	Relative to the project root. The tests override these to a temp folder.
api.base_url	https://fantasy.premierleague.com/api	Change only if FPL moves the API.
api.timeout_seconds / max_retries	30 / 3	Per-request timeout and retry count (back-off 1 s, 2 s, 4 s).

Key	Default	Effect
api.per_player_sleep_seconds	0.15	Pause between per-player calls. Lower is faster but ruder; 0.15 keeps the refresh under four minutes without tripping rate limits.

9. Terminology sheet — FPL terms

Term	Meaning	Where it appears
Gameweek (GW)	One round of fixtures, usually a weekend. 38 per season. The API calls it an <i>event</i> .	<code>gameweek</code> everywhere; <code>events</code> in <code>raw</code>
Deadline	The cut-off (usually 90 minutes before the first kick-off) after which your team for that gameweek is locked.	<code>deadline_ts</code> , report header
Entry / manager	An FPL team (yours is entry 234865, "João Mama"). The person running it is the manager.	<code>entry_id</code> , <code>raw.entry</code>
Element / player	The API's word for a footballer. <code>element_type</code> is his position.	<code>player_id</code> in <code>staging</code> = <code>element / id</code> in <code>raw</code>
Positions	GKP goalkeeper, DEF defender, MID midfielder, FWD forward. Squad: 2 / 5 / 5 / 3. Starting XI must have 1 GKP, 3–5 DEF, 2–5 MID, 1–3 FWD.	<code>position_code</code> , <code>stg.positions</code>
Price / now_cost	A player's current price. The API stores tenths of £m (55 = £5.5m). Prices move by £0.1m based on transfer volume.	<code>price_m</code> , <code>now_cost</code>
Bank	Unspent budget. Total budget is £100.0m at the start.	<code>bank_m</code>
Squad value	What your 15 players are currently worth at market price.	<code>squad_value_m</code>
Selling price	What you actually get when you sell: purchase price plus half of any rise (rounded down). Not exposed by the public API — the tool uses market price as a proxy (Phase 2 fix).	Limitation noted in §4 of the report
Free transfer (FT)	One player swap per gameweek at no cost. Unused FTs bank up to 5.	Report §1, <code>squad.py</code>
Hit	A transfer beyond your free ones costs 4 points ("taking a -4").	Report §4 guidance
Captain (C) / vice (VC)	Captain's points count double. If he doesn't play, the vice is doubled instead.	<code>is_captain</code> , <code>multiplier</code> (2 = C, 3 = triple captain)
Bench / auto-sub	Picks 12–15. If a starter doesn't play, the first eligible bench player (in bench order) comes in automatically.	<code>squad_position</code> 12–15, <code>points_on_bench</code>
Chips	One-off boosts: Wildcard (unlimited free transfers for one GW, permanent changes), Free Hit (unlimited transfers for one GW, then the squad reverts), Bench Boost (all 15 score), Triple Captain (captain ×3). Chips don't consume banked FTs.	<code>active_chip</code> , <code>raw.entry_chips</code>
Form	The API's own metric: average points per match over the last 30 days. 0 if no matches in that window.	<code>form</code> / <code>form_signal</code>
Points per game (ppg)	Season total points divided by matches with at least one minute.	<code>points_per_game</code> / <code>season_ppg</code>
Ownership / selected by %	Share of all FPL managers who own the player.	<code>selected_by_pct</code>
Differential	A low-ownership player. If he hauls, you gain on the field; if he blanks, you lose little relative to it.	<code>mart_differentials</code> , threshold 10%

Term	Meaning	Where it appears
Template	The set of very highly owned players most managers have. The opposite of a differential.	Ownership columns
Haul / blank	Haul: a big score (10+). Blank: 0–2 points. Also "blank gameweek": a GW in which a club has no fixture.	—
Double gameweek (DGW)	A club plays twice in one GW (rescheduled fixtures). Its players can score twice.	<code>fixtures_next_gw = 2</code> ; two rows in the outlook
Blank gameweek (BGW)	A club has no fixture in a GW (cup clashes). Its players score 0.	<code>fixtures_next_gw = 0</code>
FDR	Fixture Difficulty Rating, FPL's 1 (easiest) to 5 (hardest) rating of an upcoming match from one team's perspective.	<code>fdr</code> , <code>home_team_fdr</code> , <code>away_team_fdr</code>
Fixture swing / run	A stretch of easy (or hard) fixtures. Managers transfer into clubs whose run is turning good.	<code>fixture_run</code> , <code>fixture_score</code>
Clean sheet (CS)	Conceding no goals. Worth 4 to GKP/DEF, 1 to MID (must play 60+ minutes).	<code>clean_sheets</code> , the $\exp(-xGC)$ term
Bonus points (BPS)	The top three players in each match by the Bonus Points System get 3/2/1 extra points. <code>bps</code> is the raw system score; <code>bonus</code> is the points actually awarded.	<code>bonus</code> , <code>bps</code>
xG	Expected goals: the probability, summed over shots, that an average player scores from each chance. A player with 0.5 xG had chances "worth" half a goal.	<code>xg</code> , <code>xg_per_90</code>
xA	Expected assists: the xG of shots the player's passes created.	<code>xa</code> , <code>xa_per_90</code>
xGI	Expected goal involvements = xG + xA. The single cleanest attacking signal.	<code>xgi</code> , <code>xgi_per_90</code>
xGC	Expected goals conceded by the player's team while he was on the pitch. Low is good for defenders and keepers.	<code>xgc</code> , <code>xgc_per_90</code>
per 90	A rate normalised to 90 minutes so players with different minutes compare fairly. Beware small samples.	All <code>*_per_90</code> columns
ICT index	FPL's composite of Influence, Creativity and Threat. Older-style metric, kept for reference.	<code>ict_index</code>
Defensive contribution (DC)	2025/26 rule: defenders get 2 points for 10+ clearances/blocks/interceptions/tackles in a match; midfielders and forwards for 12+ including recoveries.	<code>defensive_contribution</code> (not yet modelled)
Status codes	a available, d doubtful, i injured, s suspended, u unavailable (left the league), n not in squad.	<code>status</code>
Chance of playing	FPL's stated % chance for a flagged player (25/50/75). NULL means no flag, i.e. fully fit.	<code>chance_of_playing_next_round</code>
ep_next (API)	FPL's own expected-points figure for the next gameweek. Kept as a benchmark for our model.	<code>api_ep_next</code>
Expected points (EP)	<i>Our</i> modelled points, per fixture, per gameweek or over the horizon. See section 5.	<code>expected_points</code> , <code>ep_next</code> , <code>ep_horizon</code>

Term	Meaning	Where it appears
Horizon	The number of upcoming gameweeks the model looks across (default 5).	horizon_gameweeks , horizon_index
Enabler	A cheap, reliable player whose low price frees budget for premiums. Found in the "EP per £m" table.	ep_horizon_per_m
Premium	An expensive (£9m+) high-ceiling player.	—
Effective ownership (EO)	Ownership adjusted for captaincy among your rivals. Not yet modelled (Phase 5).	—

10. Terminology sheet — data and engineering terms

Term	Meaning in this project
API / endpoint	A web address that returns data as JSON. The FPL API has several endpoints (<code>bootstrap-static/</code> , <code>fixtures/</code> ...); each is one HTTP GET.
JSON	The text format the API returns: nested objects (<code>{}</code>) and arrays (<code>[]</code>). Stored verbatim in snapshots.
Snapshot	One dated folder under <code>data/raw/</code> holding everything pulled in one refresh, plus a manifest. Immutable.
Manifest	<code>manifest.json</code> in a snapshot: when it was pulled, which GW was current/next, row counts, whether it carries player history.
Immutable	Written once, never modified. Raw snapshots are immutable so a build can be reproduced exactly.
Idempotent	Running it twice gives the same result as once. <code>append_history</code> is idempotent thanks to a primary key + <code>INSERT OR IGNORE</code> .
Landing zone / raw layer	Where data lands untouched. Here, the snapshot folders and the <code>raw</code> schema.
Schema-on-read	Types are inferred when the data is read (DuckDB's JSON reader), not enforced when it is written. Raw uses this; staging then fixes the types explicitly ("schema-on-write").
Staging	The cleaning layer: one view per raw table; rename, cast, unpivot. No business logic. Prefixed <code>stg_</code> .
Mart	An analytics-ready table built for a purpose (captaincy, transfers). Prefixed <code>mart_</code> .
Grain	What one row represents ("one row per player per fixture"). Every model comment states it. Getting the grain wrong is the classic cause of double counting.
Warehouse	The DuckDB file with its four schemas. "Building the warehouse" = <code>fpl build</code> .
DuckDB	An embedded analytical database: one file, no server, SQL close to Snowflake's, reads JSON/Parquet directly, returns pandas DataFrames.
Schema (database)	A namespace inside the database: <code>raw</code> , <code>history</code> , <code>staging</code> , <code>marts</code> . Referenced as <code>schema.table</code> .
View vs table (materialisation)	A view is a stored query, recomputed on every read (cheap, always fresh). A table is stored data (fast to read, frozen until rebuilt). Staging = views, marts = tables.
CTE	Common Table Expression: <code>WITH name AS (SELECT ...)</code> . Named intermediate steps that make a long query readable. Used throughout the marts.
Window function	<code>RANK() OVER (PARTITION BY ... ORDER BY ...)</code> : a calculation across related rows without collapsing them. Gives <code>rank_in_position</code> , <code>candidate_rank</code> .
QUALIFY	Snowflake/DuckDB clause that filters on a window function's result (<code>QUALIFY candidate_rank <= 5</code>) without a wrapping subquery.
TRY_CAST	Cast that returns NULL instead of erroring on bad input. Used for every string-typed number from the API.
UNNEST	Turns an array of objects into rows. How raw tables are made from <code>bootstrap_static.json</code> .

Term	Meaning in this project
Unpivot	Reshaping wide to long — <code>stg_team_fixtures</code> turns one fixture row (home, away) into two team rows.
LEFT JOIN	Keep every row from the left table even when the right has no match (fills NULL). How marts tolerate an empty history table.
Model (SQL)	One <code>.sql</code> file under <code>sql/</code> that becomes one view or table. dbt vocabulary.
ref() / var()	Jinja helpers in the SQL files. <code>ref('stg_players')</code> resolves to <code>staging.stg_players</code> ; <code>var('horizon_gameweeks')</code> injects a setting. Copied from dbt so a migration is mechanical.
Jinja	A templating language: text with <code>{{ placeholders }}</code> and <code>{% logic %}</code> . Used for the SQL models and the report template.
dbt	A popular tool that runs SQL models with dependency tracking, tests and docs. Not used yet; the runner mimics its conventions (Phase 3.5 option).
Lineage	Which model feeds which. Drawn in section 2.
pandas DataFrame	Python's in-memory table. What <code>query()</code> returns and what the report converts to Markdown.
TOML	The config file format (<code>key = value</code> under <code>[sections]</code>). Typed: <code>5</code> is an int, <code>0.45</code> a float.
Environment variable	A named value in the shell (<code>FPL_ANALYSIS_HORIZON_GAMEWEEKS=3</code>). Overrides a setting for one process without editing the file.
CLI	Command-line interface: the <code>fpl</code> command and its sub-commands.
Virtual environment (venv)	A private Python installation for this project (<code>.venv/</code>). Activate it (<code>source .venv/bin/activate</code>) so <code>fpl</code> and the packages are found.
Editable install	<code>pip install -e .</code> : installs the package as a link to <code>src/</code> , so code edits take effect without reinstalling.
Kernel (Jupyter)	The Python process a notebook runs in. Choose the <code>.venv</code> kernel so the project's packages are available.
Markdown	Plain-text formatting (<code># heading</code> , <code> tables </code>). The report format; renders in GitHub and VS Code.
pytest / CI	The test runner, and Continuous Integration: GitHub Actions runs lint + tests on every push.
ruff	A fast Python linter/formatter. <code>ruff check .</code> must pass in CI.
User-Agent	An HTTP header identifying the client. Set to a browser-like string so Cloudflare doesn't block the requests.
Back-off / retry	On a transient failure, wait and try again with growing delays (1, 2, 4 s).
Backtest	Comparing what the model predicted before a gameweek with what actually happened. Phase 2 deliverable.
Integer programme (ILP / MILP)	An optimisation where the unknowns are whole numbers (here 0/1: in the squad or not) under linear constraints. The chip squads are solved this way, exactly, with SciPy's HiGHS solver.
Poisson	The probability distribution for counts of rare events. $\exp(-\lambda)$ is the chance of zero events when λ is expected — used for clean-sheet probability.

11. Field glossary — the columns you will actually query

Naming rules (apply everywhere)

- *_id integer keys. `player_id` is the API's `element / id`; `team_id` is the API's `team`.
- *_m money in £ millions (API tenths ÷ 10). *_pct percentages 0–100.
- *_ts timestamps with timezone. `is_*` booleans. *_per_90 per-90-minute rates.
- `ep_*` our modelled expected points; `api_ep_*` FPL's own figure.
- `snapshot_id` on every row: which refresh it came from.

staging.stg_players (one row per player)

Column	Type	Meaning
<code>player_id</code> , <code>player_code</code>	int	API id (changes per season) and code (stable across seasons)
<code>web_name</code> , <code>first_name</code> , <code>second_name</code>	text	Display name as on the site, full names
<code>team_id</code> , <code>position_id</code>	int	Join to <code>stg_teams</code> / <code>stg_positions</code>
<code>status</code>	text	<code>a / d / i / s / u / n</code> (see §9)
<code>chance_of_playing_next_round</code>	int	NULL = fit; 0/25/50/75 = flagged
<code>news</code> , <code>news_added_ts</code>	text, ts	The injury/suspension text and when it was posted
<code>now_cost</code> , <code>price_m</code>	int, double	Price in tenths and in £m
<code>price_change_since_start_m</code> , <code>price_change_this_gw_m</code>	double	Movement since season start / this gameweek
<code>selected_by_pct</code>	double	Ownership %
<code>transfers_in_event</code> , <code>transfers_out_event</code> , <code>net_transfers_event</code>	int	Transfer volume this gameweek (price-change pressure)
<code>total_points</code> , <code>last_gw_points</code>	int	Season total; last gameweek's score
<code>points_per_game</code> , <code>form</code>	double	Season ppg; API form (last 30 days)
<code>api_ep_next</code> , <code>api_ep_this</code>	double	FPL's own expected points next / this GW
<code>value_form</code> , <code>value_season</code>	double	FPL's points-per-£m metrics
<code>minutes</code> , <code>starts</code>	int	Season minutes and starts
<code>goals_scored</code> , <code>assists</code> , <code>clean_sheets</code> , <code>goals_conceded</code> , <code>own_goals</code> , <code>penalties_saved</code> , <code>penalties_missed</code> , <code>yellow_cards</code> , <code>red_cards</code> , <code>saves</code> , <code>bonus</code> , <code>bps</code>	int	Season counting stats
<code>defensive_contribution</code>	int	Season DC total (2025/26 rule)
<code>influence</code> , <code>creativity</code> , <code>threat</code> , <code>ict_index</code>	double	FPL's ICT components

Column	Type	Meaning
xg, xa, xgi, xgc	double	Season expected goals / assists / involvements / conceded
xg_per_90, xa_per_90, xgi_per_90, xgc_per_90, saves_per_90, clean_sheets_per_90, starts_per_90, defensive_contribution_per_90	double	Per-90 rates
penalties_order, corners_and_indirect_freekicks_order, direct_freekicks_order	int	Set-piece pecking order (1 = first choice); NULL = not a taker
dreamteam_count	int	Times in the weekly Dream Team

marts.mart_player_horizon (one row per player) — the ranking table

Column	Meaning
player_id, web_name, team_id, team_short_name, position_id, position_code, price_m, selected_by_pct, status, news, chance_of_playing_next_round	Identity and current state, carried from staging
is_injury_risk	TRUE if chance of playing is set and below the threshold
total_points, minutes, starts, season_ppg, form_signal, xgi_per_90, xgc_per_90	The raw signals
xgi_points_rate	Points per 90 implied by underlying stats (section 5)
availability	0–1: chance × start share × status
api_ep_next	FPL's own next-GW expected points, for comparison
last5_points_avg	Average points over the last five matches (NULL without history)
fixtures_in_horizon, fixtures_next_gw	Count of fixtures in the horizon / next GW (0 = blank, 2 = double)
avg_fdr, fixture_run, next_fixture_label	Average difficulty; readable run; next opponent label
ep_next, ep_horizon	Modelled EP next GW and over the horizon
ep_horizon_per_m	$ep_horizon \div price_m$ — value metric
rank_overall, rank_in_position	Rank by ep_horizon across all players / within position

marts.mart_player_expected_points (one row per player per fixture) — the audit trail

Column	Meaning
gameweek, horizon_index, fixture_id, opponent_short_name, is_home, fdr, fixture_label	Which fixture. horizon_index 1 = next GW

Column	Meaning
form_used, season_ppg, xgi_used	The three inputs <i>after</i> the guards (form 0 → ppg; untrusted xGI → ppg)
base_rate	The weighted blend
availability, fdr_multiplier, home_multiplier	The three scalings
expected_points	The product, rounded to 2 dp

marts.mart_squad (one row per pick)

Column	Meaning
picks_gameweek	The gameweek these picks are for (the current one at pull time)
squad_position, is_starter	1–11 starting XI in formation order; 12–15 bench order
is_captain, is_vice_captain, multiplier	Armband; multiplier 0 benched, 1 playing, 2 captain, 3 triple captain
active_chip, bank_m, squad_value_m	Chip in play (NULL if none), bank and value — same on every row
(all mart_player_horizon columns)	The player's modelled view

marts.mart_transfer_candidates (one row per out/in pair)

Column	Meaning
out_player_id, out_web_name, out_team, position_code, out_price_m, out_ep_horizon, out_ep_next, out_is_injury_risk	Your player
bank_m	Bank before the swap
in_player_id, in_web_name, in_team, in_price_m, in_selected_by_pct, in_fixture_run, in_form, in_xgi_per_90, in_ep_next, in_ep_horizon	The candidate
ep_horizon_gain, ep_next_gain	in – out, over the horizon and next GW
bank_after_m	out price + bank – in price (market price as selling price)
candidate_rank	1–5 within the outgoing player, by ep_horizon_gain

marts.mart_team_fixture_summary (one row per team)

Column	Meaning
team_id, team_short_name, team_name, league_position	Identity
fixtures_in_horizon, home_fixtures, fixtures_next_gw	Counts
avg_fdr	Mean difficulty over the horizon (lower is easier)
fixture_score	Sum of fdr_multiplier × home_multiplier across the horizon (higher is better; doubles score more)
fixture_run, next_fixture_label, fixture_rank	Readable run; next opponent; rank by score

history.player_snapshot (one row per player per refresh)

Column	Meaning
snapshot_id, snapshot_ts, gameweek	Which refresh, when, which GW was current
player_id, team_id, position_id	Identity
now_cost, selected_by_percent, form, total_points, minutes, status, chance_of_playing_next_round, transfers_in_event, transfers_out_event	The facts worth tracking over time. Two or more snapshots give price and ownership curves

12. Raw API fields — units and gotchas

Raw tables keep the API's own column names. The ones that trip people up:

Field	Gotcha
<code>now_cost</code> , <code>bank</code> , <code>value</code> , <code>*_cost</code>	Tenths of £m. 55 means £5.5m. Staging divides by 10.
<code>form</code> , <code>points_per_game</code> , <code>selected_by_percent</code> , <code>ep_next</code> , <code>ep_this</code> , <code>ict_index</code> , <code>influence</code> , <code>creativity</code> , <code>threat</code> , <code>expected_goals</code> , <code>expected_assists</code> , <code>expected_goal_involvements</code> , <code>expected_goals_conceded</code> , <code>value_form</code> , <code>value_season</code>	Sent as strings . Compare or sort them in raw and you get text ordering ("10.0" < "9.0"). Always query staging, where they are DOUBLE.
<code>chance_of_playing_next_round</code>	NULL for a fit player, not 100.
<code>element</code> / <code>id</code> / <code>code</code>	<code>element</code> in history tables = <code>id</code> in players = our <code>player_id</code> . <code>code</code> is the cross-season identifier.
<code>element_type</code>	Position: 1 GKP, 2 DEF, 3 MID, 4 FWD.
<code>team</code> vs <code>team_code</code>	<code>team</code> is the 1–20 id used in fixtures; <code>team_code</code> is the stable club code.
<code>event</code>	Gameweek number. NULL on a fixture means unscheduled.
<code>team_h_difficulty</code> / <code>team_a_difficulty</code>	Difficulty <i>for</i> the home / away team, not <i>of</i> them.
<code>is_current</code> / <code>is_next</code>	Exactly one gameweek each during the season; both false pre-season; <code>is_next</code> false after GW38.
<code>multiplier</code> (picks)	0 benched, 1 playing, 2 captain, 3 triple captain.
<code>position</code> (picks)	1–11 starters in formation order, 12–15 bench order. Not the football position.
<code>kickoff_time</code> , <code>deadline_time</code>	ISO-8601 strings in UTC (2026-09-19T17:30:00Z). Staging casts to TIMESTAMPTZ.
<code>history_past</code> rows	Carry <code>element_code</code> , not <code>element</code> ; ingest adds <code>element</code> so they join.
<code>defensive_contribution</code> , <code>price_change_*</code> , <code>scout_risks</code>	Added in 2025/26. Present in 2026/27. Not all modelled yet.
<code>entry/{id}/event/{gw}/picks/</code> , <code>entry/{id}/transfers/</code> for the <i>upcoming</i> GW	Hidden until the deadline. Picks return 404 and pending transfers are omitted, so before a deadline the pipeline sees last week's squad. State your real squad in <code>config/squad_override.toml</code> (set <code>enabled = true</code>); the report header then says "squad from <code>squad_override.toml</code> ". Switch it off after the deadline.

13. Troubleshooting

Symptom	Cause	Fix
Warehouse ... does not exist yet in a notebook	You haven't built yet.	<code>fpl run</code> (or <code>fpl build</code> if a snapshot exists), then re-run the cell.
No snapshots under <code>data/raw</code>	No refresh has completed, or a folder has no manifest (old version, interrupted).	<code>fpl refresh</code> . Folders without a manifest are ignored.
<code>ModuleNotFoundError: fpl_analysis / pandas</code>	Wrong interpreter or venv not activated.	Terminal: <code>source .venv/bin/activate</code> . Notebook: select the <code>.venv</code> kernel.
Could not set lock on file from DuckDB	Another process (a notebook with an open <code>connect()</code>) holds the file.	Close the connection or restart the kernel, then retry. <code>query()</code> closes after each call and is safe.
Refresh is slow / stops with "consecutive failures"	Cloudflare is rate-limiting the per-player endpoint.	Wait a few minutes; or <code>fpl refresh --no-history</code> for a fast, history-less snapshot.
Report shows last week's squad after I made transfers	The API hides upcoming-GW picks until the deadline.	Fill in <code>config/squad_override.toml</code> , set <code>enabled = true</code> , <code>fpl build && fpl report</code> .
Report shows "No entry data"	Snapshot was taken with <code>--no-entry</code> , or the entry endpoint failed.	Refresh again without the flag; check <code>entry_id</code> in settings.
Free transfers look wrong	A rule edge case (chip refund, rule change).	Set <code>free_transfers_override</code> ; report it so the estimator can be fixed.
A staging view fails after a refresh	The API renamed or removed a field.	Check <code>DESCRIBE raw.players</code> ; fix the one line in the staging file; tests will catch regressions.
A player's EP looks absurd	Usually minutes/availability: a one-cameo wonder with a big per-90 rate, or a star with 0 form after injury.	Look him up in <code>mart_player_form: rates_are_trusted, start_share, form_signal</code> explain it. If the guards missed, tune the thresholds.

14. Known limitations and where the plan goes next

Limitation	Effect	Planned fix
Pre-deadline squad is invisible to the public API	On deadline day the model sees last week's team unless you tell it otherwise.	Now: <code>config/squad_override.toml</code> . Phase 2.0: authenticated <code>my-team pull</code> (also gives the real free-transfer count).
Selling price = market price	Transfer affordability can be off by £0.1–0.5m for players who have risen since you bought them.	Phase 2.1: derive purchase price from transfer history and GW1 prices.
Crude minutes model	New signings and returning players are under-rated; sub appearances ignored.	Phase 2.2: weight last-5 starts; treat 75% flags probabilistically.
Defensive-contribution points not modelled	Defensive midfielders slightly under-rated in the xGI component (ppg and form still capture it).	Phase 2.3.

Limitation	Effect	Planned fix
Weights are reasoned, not fitted	The 45/30/25 split may not be optimal.	Phase 2.4: backtest harness comparing pre-deadline EP to actual points each week.
One-for-one transfers only	No two-move combinations.	Phase 2.5.
No chip planner	Bench Boost / Free Hit / Triple Captain timing is manual.	Phase 3.1, once double/blank gameweeks are announced.
Single-snapshot history	Price and ownership curves need two or more refreshes.	Phase 3.2: daily 02:30 UK snapshot.

Full phase plan, cadence and risk register: `docs/PROJECT_PLAN.md` . Architecture rationale: `docs/ARCHITECTURE.md` . Model definition: `docs/SCORING_MODEL.md` . Endpoint reference: `docs/FPL_API_REFERENCE.md` .